Labo 16 — Ressources et exploitation

Théorie — ce qu'un cluster partagé exige de chaque application : dire ce qu'elle consomme, accepter un budget, pouvoir être observée et déboguée, n'avoir que les droits qu'il lui faut, et ne parler qu'à qui elle doit.

Objectifs

- Déclarer **requests** et **limits**, comprendre les classes de **QoS**, l'`OOMKilled` et le `Pending`.
- Cloisonner avec les **namespaces**, et borner avec **LimitRange** et **ResourceQuota**.
- Observer et déboguer : `kubectl top`, `events`, `logs --previous`, `describe`, `kubectl debug`.
- Lire et écrire un **RBAC** minimal : ServiceAccount, Role, RoleBinding, `kubectl auth can-i`.
- Restreindre le réseau avec une **NetworkPolicy**.

---

1. Un cluster est partagé

Jusqu'ici, votre cluster n'avait qu'un locataire : vous. En entreprise, un cluster héberge des dizaines d'applications et d'équipes sur les mêmes nœuds. Trois questions deviennent vitales : combien chaque Pod consomme-t-il (pour le placer et le protéger), qui a le droit de faire quoi (pour ne pas supprimer le Deployment du voisin), et qui peut parler à qui (pour qu'une faille dans le front ne donne pas accès à la base de la comptabilité). Le labo 10 posait ces questions pour un conteneur ; le cluster y répond avec des objets.

> **→ RAPPEL DOCKER**
> `--memory 256m --cpus 0.5` deviennent `resources.limits` ; l'OOM killer et le code `137` sont les mêmes (les Pods tournent dans les mêmes cgroups, labo 10) ; `podman stats` devient `kubectl top` ; `podman logs --previous`… n'existait pas, mais `podman logs` d'un conteneur relancé par `--restart` posait le même problème ; et `--cap-drop` / `--read-only` sont repris tels quels dans le `securityContext` du Pod.

2. Requests et limits : dire ce qu'on consomme

```
resources:
  requests: { cpu: "100m", memory: "64Mi" }     # ce que le scheduler réserve
  limits:   { cpu: "500m", memory: "256Mi" }    # ce que le cgroup plafonne
```

La **request** est une **réservation** : le scheduler ne place le Pod que sur un nœud où la somme des requests reste sous la capacité, et c'est elle qui est comptée dans `Allocated resources` de `kubectl describe node`. La **limit** est le **plafond** cgroup du labo 10 : la mémoire au-delà tue le conteneur (`OOMKilled`, code `137`, et le kubelet relance avec `CrashLoopBackOff`), le CPU au-delà est **ralenti** (*throttling*), jamais tué. `100m` = un dixième de cœur ; `64Mi` = 64 mébioctets.

Classe de QoS	Condition	Sort en cas de pression mémoire sur le nœud
`Guaranteed`	requests = limits pour CPU et mémoire	évincé en dernier
`Burstable`	des requests, des limits différentes ou partielles	évincé après les BestEffort
`BestEffort`	aucune request ni limit	évincé en premier

Le kubelet calcule la classe à partir de la spec (`status.qosClass`) ; elle décide qui est sacrifié quand un nœud manque de mémoire. Une base en `BestEffort` est une base qu'on tuera la première.

> **→ JAVA**
> La JVM voit la **limit** comme mémoire totale (labo 10) : avec `limits.memory: 256Mi` et `-XX:MaxRAMPercentage=75`, le tas fait 192 Mi, et le reste (metaspace, threads) doit tenir dans les 64 Mi restants. Un `OOMKilled` sur une API Java vient presque toujours d'un pourcentage trop ambitieux ou d'une limit trop juste. Pour le CPU, une request de `100m` suffit à tourner, mais une JVM qui démarre consomme un cœur entier pendant dix secondes : une `limit` CPU trop basse allonge le démarrage et fait échouer les probes — beaucoup d'équipes ne mettent **pas** de limit CPU.

Une request trop grande a un autre effet : le Pod reste `Pending` avec `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. Rien ne se passe tant qu'un nœud ne peut pas l'accueillir — c'est le symptôme le plus fréquent après une mise à l'échelle.

3. Namespaces, LimitRange, ResourceQuota

Le **namespace** cloisonne les noms (deux `api` dans deux namespaces), porte les droits RBAC et les quotas, et se supprime d'un bloc. L'organisation habituelle : un namespace par application et par environnement (`facturation-prod`, `facturation-recette`), ou un par équipe. `kube-system` abrite les composants du cluster ; `default` ne devrait rien contenir.

Deux objets bornent un namespace. Le **LimitRange** fixe des **valeurs par défaut** (un Pod créé sans `resources` reçoit `defaultRequest` et `default`) et des **plafonds** par conteneur (`max`) : un Pod qui demande plus est **refusé** à l'`apply` (`maximum memory usage per Container is 1Gi, but limit is 4Gi`). La **ResourceQuota** fixe un **budget total** : somme des requests, des limits, nombre de Pods, de Services, de PVC — `kubectl describe quota` montre `Used` et `Hard`. Avec une quota active, tout Pod **doit** déclarer des ressources (ou avoir un LimitRange qui les fournit).

> **K3S**
> Un nœud unique rend tout cela visible d'un coup : `kubectl describe node` montre la capacité de votre WSL (ce que `.wslconfig` lui a donné) et les requests de tous les Pods, dont ceux de `kube-system`. Sur un cluster managé, l'équipe plateforme pose LimitRange et ResourceQuota dans chaque namespace qu'elle vous confie : un Pod sans `resources` y est refusé ou doté de défauts — c'est pourquoi un manifeste d'entreprise les déclare toujours.

4. Observer et déboguer

Besoin	Commande
consommation réelle (CPU, mémoire)	`kubectl top nodes`, `kubectl top pods` (metrics-server, fourni par k3s)
ce qui s'est passé	`kubectl get events --sort-by=.lastTimestamp`, `kubectl describe pod` (section Events)
pourquoi le conteneur précédent est mort	`kubectl logs pod --previous`
explorer un conteneur sans shell	`kubectl debug pod -it --image=alpine --target=conteneur --profile=general`
copier un fichier	`kubectl cp pod:/chemin ./fichier`
tout sur un objet	`kubectl get pod -o yaml` (status, conditions, `lastState.terminated.reason`)

`kubectl debug` mérite un mot : il ajoute un **conteneur éphémère** au Pod, dans le même namespace de processus que la cible (`--target`), avec les outils que vous voulez (`alpine`, `busybox`, `nicolaka/netshoot`). C'est la réponse à l'image distroless du labo 05 : pas de shell dans l'image, un shell à côté le temps du diagnostic. Le conteneur éphémère disparaît avec le Pod et n'entre jamais dans le template.

La grille de lecture d'un incident reste celle du labo 12 : `kubectl get pods` (état, `RESTARTS`), `describe` (Events, `Last State`), `logs --previous`, `top` ; puis, pour le réseau, la chaîne du labo 13.

5. RBAC : ne donner que le nécessaire

Tout appel à l'API server est **authentifié** (certificat, jeton, OIDC) puis **autorisé** par le **RBAC** (*role-based access control*). Quatre objets :

Objet	Portée	Contenu
Role / **ClusterRole**	un namespace / tout le cluster	des règles : `resources` + `verbs` (`get`, `list`, `watch`, `create`, `update`, `patch`, `delete`)
RoleBinding / **ClusterRoleBinding**	un namespace / tout le cluster	relie un rôle à des sujets : utilisateurs, groupes, **ServiceAccounts**

Un **ServiceAccount** est l'identité d'une application dans le cluster ; chaque Pod en a un (`default` si rien n'est dit), dont le jeton est monté dans `/var/run/secrets/kubernetes.io/serviceaccount/`. Une application qui n'appelle pas l'API n'a besoin d'aucun droit — et `automountServiceAccountToken: false` lui retire même le jeton. `kubectl auth can-i <verbe> <ressource> --as=system:serviceaccount:<ns>:<nom>` teste une permission sans se reconnecter, et `--list` énumère tout ce qu'un sujet peut faire.

> **→ SÉCURITÉ**
> Les ClusterRoles livrés (`view` , `edit` , `admin` , `cluster-admin`) couvrent la plupart des besoins humains : `view` pour lire sans les Secrets, `edit` pour déployer, `admin` pour gérer le RBAC du namespace. `cluster-admin` est le root du cluster : jamais à une application, rarement à un humain. Le kubeconfig de k3s **est** `cluster-admin` — d'où le labo 11 et son `644` . Le **moindre privilège** du labo 10 s'applique : un Pod qui n'a pas besoin de l'API a un ServiceAccount vide, et un opérateur qui en a besoin a un Role listant exactement ses ressources.

6. NetworkPolicy : ne parler qu'à qui il faut

Par défaut, **tout Pod peut joindre tout Pod**, dans tous les namespaces (labo 13). Une **NetworkPolicy** sélectionne des Pods (`podSelector`) et déclare ce qui est autorisé en entrée (`ingress`) et/ou en sortie (`egress`) ; dès qu'une politique sélectionne un Pod, **tout ce qui n'est pas listé est refusé**. L'exemple du labo : seuls les Pods `app=web` peuvent joindre `app=api` sur `8080` ; un client sans ce label reçoit `Connection refused` . Les politiques se cumulent (union des autorisations), et la pratique courante commence par une politique *deny-all* par namespace, puis des autorisations explicites.

> **→ RÉSEAU**
> La NetworkPolicy est un objet de l'API ; c'est le **CNI** qui l'applique (ou non). Flannel seul ne le fait pas ; k3s embarque un contrôleur de politiques (kube-router) qui programme des règles de filtrage sur chaque nœud, Calico et Cilium font de même avec plus de finesse (niveau 7, FQDN). Sur un cluster dont le CNI ignore les politiques, l'objet est accepté et **sans effet** : vérifiez avec un test, comme au labo. Les règles sont au niveau 3/4 (IP, port) : la NetworkPolicy ne remplace pas l'authentification entre services.

7. En entreprise

- **Tout manifeste déclare** `resources` , avec des requests mesurées (`kubectl top` en charge) et des limits mémoire ; `Guaranteed` pour les bases et ce qui ne doit pas être évincé.
- **Un namespace par application et par environnement**, avec LimitRange et ResourceQuota posés par la plateforme ; les développeurs reçoivent `edit` dessus, `view` ailleurs.
- **Les applications ont un ServiceAccount dédié**, sans droit par défaut ; les opérateurs et agents (CI, GitOps) ont des rôles précis, audités.
- **Deny-all + autorisations explicites** en NetworkPolicy, au moins en production ; la base n'accepte que l'API.
- **L'observabilité** est un service de la plateforme (Prometheus, Loki, Grafana) qui lit ce que vos Pods exposent : `stdout` , `/actuator/prometheus` , les Events.

À retenir

- `requests` réservent et placent, `limits` plafonnent ; mémoire dépassée = `OOMKilled` (137), CPU dépassé = ralenti ; request impossible = `Pending` .
- QoS `Guaranteed` > `Burstable` > `BestEffort` : l'ordre dans lequel le nœud sacrifie les Pods.
- Namespace = cloison ; LimitRange = défauts et plafonds par conteneur ; ResourceQuota = budget du namespace.

- `top`, `events`, `describe`, `logs --previous`, `debug --target` : la trousse de diagnostic, sans rien ajouter aux images.
- RBAC = Role (quoi) + RoleBinding (qui) ; `auth can-i --as=` pour tester ; une application a un ServiceAccount sans droit.
- NetworkPolicy = liste blanche par label, appliquée par le CNI (k3s le fait) ; sans politique, tout parle à tout.

Vocabulaire

request / limit : réservation pour le scheduler / plafond cgroup. — **QoS** : `Guaranteed`, `Burstable`, `BestEffort`. — **`OOMKilled`** : conteneur tué pour dépassement mémoire (137). — **throttling** : ralentissement CPU à la limit. — **LimitRange** : défauts et plafonds par conteneur dans un namespace. — **ResourceQuota** : budget total d'un namespace. — **metrics-server** : source de `kubectl top`. — **conteneur éphémère** : conteneur de diagnostic ajouté par `kubectl debug`. — **RBAC** : Role, ClusterRole, RoleBinding, ClusterRoleBinding. — **ServiceAccount** : identité d'un Pod auprès de l'API. — **`kubectl auth can-i`** : test de permission. — **NetworkPolicy** : règles de trafic entre Pods, appliquées par le CNI.